

SQL 처리기 완전정복

크래프톤 정글 WEEK6 — 이것만 보면 수요 코딩회 준비 끝

목차

Chapter 1. C 기초 복습 — 이것만 알면 된다

1-1. 포인터

1-2. 문자열

1-3. 구조체와 열거형

1-4. 파일 입출력

Chapter 2. SQL이 뭔지 이해하기

Chapter 3. 전체 설계 — 큰 그림

Chapter 4. types.h — 데이터 설계

Chapter 5. parser.c — SQL 파싱 (핵심)

Chapter 6. storage.c — 파일 기반 DB

Chapter 7. executor.c — 실행기

Chapter 8. main.c — CLI + 전체 연결

Chapter 9. Makefile — 빌드 자동화

Chapter 10. 테스트 전략

Chapter 11. 엡지 케이스 & 에러 처리

Chapter 12. 차별화 추가 구현

Chapter 13. 발표 준비 체크리스트

C 기초 복습 — 이것만 알면 된다

SQL 처리기를 만들려면 C에서 딱 4가지만 확실히 알면 된다: 포인터, 문자열, 구조체, 파일 입출력. 나머지는 다 이 4가지 조합이다.

1-1. 포인터 — 주소를 가리키는 변수

포인터가 뭐냐? 메모리 주소를 저장하는 변수다. 그게 다다.

```
int a = 42;           // 정수 변수. 값은 42.
int* p = &a;          // 포인터 변수. a의 "주소"를 저장.

printf("%d\n", *p);   // *p = "p가 가리키는 곳의 값" = 42
```

비유하면 이렇다:

a = 아파트 101호에 사는 사람 (값: 42) p = "101호"라고 적힌 메모지 (주소를 저장) *p = 메모지 보고 101호 찾아가서 사람 만남 (값: 42) &a = 101호의 주소를 알려줌

이 프로젝트에서 포인터가 쓰이는 곳

- 1 문자열 처리 — C에서 문자열은 사실 `char*` (char 포인터)다.

```
char* sql = "INSERT INTO students VALUES (1, 'Kim', 20);";

// sql[0] = 'I', sql[1] = 'N', ... 이런 식으로 접근 가능
// sql은 문자열의 "시작 주소"를 가리키는 포인터
```

2 함수에 구조체 넘기기 — 포인터로 넘기면 원본을 직접 수정할 수 있다.

```
// 포인터 없이 넘기면: 복사본이 전달됨 (원본 변경 불가)
void bad(ParsedSQL result) {
    result.type = SQL_INSERT; // 이걸 복사본만 바뀜. 원본은 그대로.
}

// 포인터로 넘기면: 원본 주소가 전달됨 (원본 직접 수정)
void good(ParsedSQL* result) {
    result->type = SQL_INSERT; // 원본이 바뀜!
}

// 호출할 때:
ParsedSQL parsed;
good(&parsed); // &parsed = parsed의 주소를 넘김
```

핵심 규칙 3개만 기억해:

&변수 = 변수의 주소를 꺼낸다

*포인터 = 포인터가 가리키는 값을 꺼낸다

포인터->멤버 = 포인터가 가리키는 구조체의 멤버 접근 ((*포인터).멤버 의 축약)

3 파일 포인터 — FILE* 도 포인터다.

```
FILE* fp = fopen("data.csv", "r");
// fp는 "열린 파일"을 가리키는 포인터
// fgets, fprintf 등의 함수에 이 포인터를 넘겨서 파일 작업
```

1-2. 문자열 — SQL 파싱의 기반

C에는 `string` 타입이 없다. 대신 `char` 배열 + 끝에 `'\0'`이 문자열이다.

```
char name[10] = "Kim";
```

```
// 메모리에 이렇게 저장됨:
```

```
// [0] [1] [2] [3] [4] [5] [6] [7] [8] [9]
// 'K' 'i' 'm' '\0' ? ? ? ? ? ?
//                               ↑ 여기가 끝 표시 (null terminator)
```

이 프로젝트에서 필수인 문자열 함수들

1. strlen() — 문자열 길이

```
strlen("Hello"); // 5 ('\0'은 안 셈)
```

```
strlen("INSERT INTO "); // 12
```

```
// 용도: "INSERT INTO " 다음 위치로 이동할 때
```

```
const char* p = sql + strlen("INSERT INTO ");
```

```
// → sql 시작점에서 12칸 뒤로 이동 = 테이블 이름 시작점
```

2. strcmp() — 문자열 비교

```
// C에서는 == 로 문자열 비교 불가! (주소 비교가 됨)
```

```
if (str == "hello") // ❌ 이러면 안 됨
```

```
if (strcmp(str, "hello") == 0) // ✅ 이렇게 해야 함
```

```
// strcmp 반환값:
```

```
// 0 → 같다
```

```
// 양수 → str이 더 크다 (사전순)
```

```
// 음수 → str이 더 작다
```

3. strncasecmp() — 대소문자 무시하고 N글자 비교

```
// SQL은 대소문자를 안 가림: INSERT = insert = Insert
```

```
strncasecmp("INSERT INTO", "insert into", 11); // 0 (같다)
```

```
strncasecmp("SELECT", "select", 6); // 0 (같다)
```

```
// 용도: SQL 문장이 INSERT로 시작하는지 확인
```

```
if (strncasecmp(sql, "INSERT INTO", 11) == 0) {  
    // INSERT 문이다!  
}
```

4. strchr() — 문자 1개 찾기

```
char* sql = "VALUES (1, 'Kim', 20)";  
  
char* p = strchr(sql, '(');  
// p는 이제 "(1, 'Kim', 20)" 을 가리킴  
// 즉 '('가 있는 위치의 주소를 반환  
  
// 못 찾으면 NULL 반환  
char* q = strchr(sql, '@');  
// q = NULL (@ 없으니까)
```

5. strcasestr() — 부분 문자열 찾기 (대소문자 무시)

```
char* sql = "INSERT INTO students VALUES (1, 'Kim', 20);";  
  
char* p = strcasestr(sql, "VALUES");  
// p는 "VALUES (1, 'Kim', 20);" 을 가리킴  
// "values", "Values" 도 다 찾아줌
```

6. strtok() — 문자열 분리 (매우 중요!)

```
char str[] = "1, 'Kim', 20";  
  
// 첫 번째 호출: 원본 문자열 전달  
char* token = strtok(str, ",");  
// token = "1"  
  
// 두 번째부터: NULL 전달 (이전 위치에서 이어서 분리)  
token = strtok(NULL, ",");  
// token = " 'Kim'"  
  
token = strtok(NULL, ",");
```

```
// token = " 20"

token = strtok(NULL, ",");
// token = NULL (더 이상 없음)
```

strtok 주의사항:

strtok은 원본 문자열을 직접 수정한다! 심표를 '\0'으로 바꿔버림.

그래서 원본을 보존하고 싶으면 복사본을 만들어서 strtok에 넘겨야 한다.

```
char buf[1024];
strcpy(buf, original); // 복사본 만들기
strtok(buf, ",");      // 복사본을 쪼갬 (원본은 안전)
```

7. strncpy() / strcpy() — 문자열 복사

```
char dest[64];

// 전체 복사
strcpy(dest, "hello"); // dest = "hello"

// N글자만 복사 (안전함)
strncpy(dest, source, 10);
dest[10] = '\0'; // 반드시 끝에 '\0' 직접 넣어줘야!
```

8. strcspn() — 특정 문자까지의 길이

```
char line[] = "Hello World\n";
int len = strcspn(line, "\n"); // 11 ('\n' 이전까지의 길이)

// 줄바꿈 제거 트릭 (이 프로젝트에서 매우 자주 씬):
line[strcspn(line, "\n")] = '\0';
// '\n' 위치에 '\0'을 덮어써서 줄바꿈 제거
```

9. isspace() — 공백 문자 판별

```
// 스페이스, 탭, 줄바꿈 등을 공백으로 판별
isspace(' '); // 참 (1)
isspace('\t'); // 참 (1)
isspace('A'); // 거짓 (0)

// 용도: 앞쪽 공백 건너뛰기
while (isspace(*str)) str++;
// "  hello" → str이 "hello"를 가리키게 됨
```

문자열 함수 요약 치트시트:

길이 알기 → `strlen`

비교하기 → `strcmp` / `strncasecmp`

한 글자 찾기 → `strchr`

부분 문자열 찾기 → `strcasestr`

쪼개기 → `strtok`

복사하기 → `strcpy` / `strncpy`

공백 판별 → `isspace`

1-3. 구조체와 열거형

구조체(struct)는 여러 변수를 하나로 묶는 것이다. Python의 class, JS의 object와 비슷하다.

```
// 정의
typedef struct {
    char name[64];
    int age;
    float gpa;
} Student;
```

```
// 사용
Student s;
strcpy(s.name, "Kim"); // 문자열은 strcpy로 넣기
s.age = 20;             // 숫자는 = 으로 넣기
s.gpa = 3.8;

printf("%s is %d years old\n", s.name, s.age);
// 출력: Kim is 20 years old
```

포인터로 접근할 땐 -> 를 쓴다:

```
Student* p = &s;
printf("%s\n", p->name); // "Kim"
p->age = 21;             // 원본의 age가 21로 바뀜
```

열거형(enum)은 상수에 이름을 붙이는 것이다.

```
// 정의
typedef enum {
    SQL_INSERT,    // 내부적으로 0
    SQL_SELECT,    // 내부적으로 1
    SQL_UNKNOWN    // 내부적으로 2
} SqlType;

// 사용: 숫자 대신 이름으로 쓰니까 코드가 읽기 쉬워짐
SqlType type = SQL_INSERT;

if (type == SQL_INSERT) {
    // INSERT 처리
}

// switch문과 궁합이 좋다:
switch (type) {
    case SQL_INSERT: /* ... */ break;
    case SQL_SELECT: /* ... */ break;
    default: /* 에러 */ break;
}
```


Q. typedef는 뭔가요?

typedef 는 타입에 별명을 붙이는 것이다. typedef 없으면 매번 `struct ParsedSQL` 이라고 써야 하는데, typedef 쓰면 그냥 `ParsedSQL` 이라고만 쓸 수 있다. 편의 기능이다.

1-4. 파일 입출력

이 프로젝트의 절반이 파일 읽기/쓰기다. 패턴이 항상 같으니까 외워두면 좋다.

파일 열기/닫기 기본 패턴

```
// 1. 열기
FILE* fp = fopen("파일경로", "모드");

// 2. 열기 실패 체크 (파일 없으면 NULL 반환)
if (fp == NULL) {
    fprintf(stderr, "파일 열기 실패!\n");
    return -1;
}

// 3. 파일 작업 (읽기 or 쓰기)
// ...

// 4. 닫기 (반드시!)
fclose(fp);
```

파일 열기 모드 (이것만 알면 됨)

모드	의미	이 프로젝트에서의 용도
"r"	읽기 전용	SELECT할 때 CSV 읽기, SQL 파일 읽기

"w"	쓰기 (기존 내용 삭제)	파일 새로 만들 때
"a"	추가 (기존 내용 뒤에 이어쓰기)	INSERT할 때 CSV에 행 추가

파일에서 읽기 — fgets()

```
FILE* fp = fopen("data.csv", "r");

char line[1024]; // 한 줄을 담을 버퍼

// fgets: 한 줄씩 읽음. 파일 끝이면 NULL 반환.
while (fgets(line, sizeof(line), fp) != NULL) {
    // line에 한 줄이 들어있음 (줄바꿈 포함!)
    line[strcspn(line, "\n")] = '\0'; // 줄바꿈 제거
    printf("읽은 줄: %s\n", line);
}

fclose(fp);
```

fgets 동작 원리:

파일에서 한 줄을 읽어서 `line` 배열에 넣는다.

줄바꿈(`\n`)까지 포함해서 읽는다. 그래서 직접 제거해줘야 한다.

`sizeof(line)` 은 버퍼 크기. 이것보다 긴 줄은 잘림 (안전장치).

파일에 쓰기 — fprintf()

```
FILE* fp = fopen("data.csv", "a"); // "a" = 끝에 추가

// printf랑 사용법 동일. 첫 인자가 파일 포인터일 뿐.
fprintf(fp, "%d,%s,%d\n", 1, "Kim", 20);
// 파일에 "1,Kim,20\n" 이 추가됨

fclose(fp);
```

Q. printf vs fprintf 차이는?

`printf` 는 화면(터미널)에 출력. `fprintf` 는 지정한 파일에 출력. `fprintf`의 첫 인자로 `stdout` 을 넘기면 `printf`와 동일하게 동작한다. `stderr` 를 넘기면 에러 출력으로 쓸 수 있다.

CHAPTER 2

SQL이 뭔지 이해하기

SQL(Structured Query Language)은 데이터베이스에 명령을 내리는 언어다. 우리가 구현할 건 이 중에 딱 2개:

INSERT — 데이터 넣기

```
INSERT INTO 테이블이름 VALUES (값1, 값2, 값3);
```

실제 예시:

```
INSERT INTO students VALUES (1, 'Kim', 20);
-- students 테이블에 {id:1, name:'Kim', age:20} 추가

INSERT INTO students VALUES (2, 'Lee', 22);
-- students 테이블에 {id:2, name:'Lee', age:22} 추가
```

이걸 우리 프로그램에서는 CSV 파일에 한 줄 추가로 구현한다:

`students.csv` 파일: | 1,Kim,20 | ← 첫 번째 INSERT 결과 | 2, Lee, 22 | ←
두 번째 INSERT 결과

SELECT — 데이터 조회

```
SELECT * FROM 테이블이름;
```

실제 예시:

```
SELECT * FROM students;  
-- students 테이블의 모든 데이터를 보여줘  
-- * 은 "전부 다"라는 뜻
```

이걸 우리 프로그램에서는 CSV 파일을 읽어서 화면에 출력으로 구현한다.

SQL 문장의 구조

```
INSERT INTO students VALUES (1, 'Kim', 20);  
----- 키워드 테이블명 키워드 값들  
SELECT * FROM students;  
----- 키워드 컬럼 키워드 테이블명
```

우리가 할 일의 본질:

SQL 문자열에서 키워드 사이사이에 있는 테이블명 과 값들 을 뽑아내는 것. 이게 파싱이다.

CHAPTER 3

전체 설계 — 큰 그림

프로그램 사용 방법 — 대화형 CLI (REPL)

SQL 파일을 만들어서 넘기는 게 아니라, 프로그램을 실행하면 프롬프트가 뜨고, 거기에 SQL을 직접 타이핑하는 방식이다. 마치 MySQL이나 SQLite 셸처럼.

```
# 프로그램 실행
$ ./sql_processor

# 프롬프트가 뜬다. 여기에 SQL을 직접 입력.
db> insert into students values (1, 'Kim', 20);
    Inserted into 'students'
db> insert into students values (2, 'Lee', 22);
    Inserted into 'students'
db> select * from students;
    Row 0: 1 | Kim | 20
    Row 1: 2 | Lee | 22
db> .exit
Bye!
```

핵심 차이점:

이전 버전: `./sql_processor input.sql` (파일을 넘기는 방식)

지금 버전: `./sql_processor` (실행 후 프롬프트에서 직접 입력)

이 방식을 **REPL (Read-Eval-Print Loop)**이라고 부른다.

Read: 사용자 입력을 읽고 → Eval: 파싱+실행하고 → Print: 결과를 출력하고 → Loop: 반복.

추가로, .sql 파일 실행도 지원하면 금상첨화:

`./sql_processor test.sql` 처럼 인자가 있으면 파일 모드,

인자 없이 `./sql_processor` 만 치면 대화형 모드로 동작하게 하면 된다.

이건 Chapter 8 main.c에서 구현한다.

프로그램 내부 흐름

```
| main.c | |
"db> " 프롬프트를 띄우고, 사용자가 SQL을 입력할 때까지 대기 | | 입력을 받으면 아래 과정을 실행하고, 다
시 프롬프트로 돌아온다 | | ".exit" 입력 시 종료 |
| SQL 문장 1줄
(사용자 입력) ▼
```


비유하면: .h = 메뉴판 (뭐가 있는지 보여줌), .c = 주방 (실제로 만들)

CHAPTER 4

types.h — 데이터 설계

모든 파일이 공유하는 구조체와 상수를 정의한다. 프로그램의 "설계도".

```
#ifndef TYPES_H          // 중복 include 방지 (관례적으로 항상 씬)
#define TYPES_H

// — 상수 정의 —
#define MAX_COLUMNS      32    // 한 테이블 최대 컬럼 수
#define MAX_VALUE_LEN    256   // 하나의 값 최대 길이
#define MAX_TABLE_NAME   64    // 테이블 이름 최대 길이
#define MAX_SQL_LEN      1024  // SQL 한 줄 최대 길이

// — SQL 종류 —
typedef enum {
    SQL_INSERT,      // 0
    SQL_SELECT,      // 1
    SQL_UNKNOWN      // 2
} SqlType;

// — 파싱 결과 구조체 —
typedef struct {
    SqlType type;                // 어떤 SQL인지
    char    table_name[MAX_TABLE_NAME]; // 테이블 이름

    // INSERT할 때 쓰는 필드
    char    values[MAX_COLUMNS][MAX_VALUE_LEN]; // 값 배열
    int     value_count;           // 값 개수

    // SELECT할 때 쓰는 필드
```

```

    char    columns[MAX_COLUMNS][MAX_VALUE_LEN]; // 컬럼 목록
    int     column_count;                        // 컬럼 개수
} ParsedSQL;

#endif

```

ParsedSQL이 채워지는 과정 시각화

SQL: **INSERT INTO students VALUES (1, 'Kim', 20);** ↓ 파싱 후

	ParsedSQL	type = SQL_INSERT
table_name = "students"	values[0] = "1"	values[1] = "Kim"
values[2] = "20"	value_count = 3	columns = (사용 안 함)
	column_count = 0	

SQL: **SELECT * FROM students;** ↓ 파싱 후

	ParsedSQL	type = SQL_SELECT
table_name = "students"	columns[0] = "*" (사용 안 함)	column_count = 1
values =	value_count = 0	

Q. #ifndef / #define / #endif는 뭔가요?

Include Guard라고 부른다. 같은 헤더가 여러 번 include되면 "이미 정의됨" 에러가 난다. 이것 방지하는 관례적인 패턴이다.

```

#ifndef TYPES_H = "TYPES_H가 정의 안 되어 있으면"
#define TYPES_H = "TYPES_H를 정의해" (다음번엔 통과 못 함)
#endif = "조건 끝"

```

CHAPTER 5

parser.c — SQL 파싱 (핵심 중의 핵심)

이 프로젝트에서 가장 어렵고, 가장 중요한 파일이다. 시간을 가장 많이 쓸 곳.

parser.h (헤더 파일)

```
#ifndef PARSER_H
#define PARSER_H

#include "types.h"

// 외부에서 호출할 함수 선언 (메뉴판)
int parse_sql(const char* sql, ParsedSQL* result);

#endif
```

필요한 헤더들

```
#include <stdio.h>    // printf, fprintf
#include <string.h>    // strlen, strcmp, strchr, strtok, strncpy, ...
#include <ctype.h>     // isspace
#include "types.h"
#include "parser.h"
```

보조 함수: trim (공백 제거)

SQL에서 값 사이에 공백이 들어갈 수 있다. " 'Kim' " → "'Kim'" 으로 정리해야 한다.

```
// 문자열 앞뒤 공백을 제거하는 함수
// 원리: 앞쪽은 포인터를 이동, 뒤쪽은 '\0'을 덮어쓰
static char* trim(char* str) {
    // 1단계: 앞쪽 공백 건너뛰기
    while (isspace(*str)) str++;
    // " hello " → str이 "hello "를 가리킴

    // 빈 문자열이면 바로 리턴
    if (*str == '\0') return str;

    // 2단계: 뒤쪽 공백 제거
    char* end = str + strlen(str) - 1;
```

```

//   end는 마지막 문자를 가리킴
while (end > str && isspace(*end)) end--;
//   end가 공백이 아닌 마지막 문자까지 이동
*(end + 1) = '\0';
//   그 다음 위치에 '\0' 덮어쓰기 = 문자열 끝 당기기

return str;
}

```

trim 동작 예시: 입력: " **hello world** " 1단계 (앞쪽 공백 건너뛰기): [][][h][e][l][l][o][][w][o][r][l][d][][][\0] ↑ str이 여기를 가리킴 2단계 (뒤쪽 공백 제거): [h][e][l][l][o][][w][o][r][l][d][\0] ↑ 여기에 '\0' 덮어쓰기 결과: "**hello world**"

Q. static이 함수 앞에 붙으면?

static 이 함수 앞에 붙으면 "이 .c 파일 안에서만 쓸 수 있는 함수"라는 뜻이다. 다른 파일에서 호출 불가. trim은 parser.c 안에서만 쓰니까 static을 붙인 것.

INSERT 파싱 — 한 줄씩 해부

```

static int parse_insert(const char* sql, ParsedSQL* result) {
    result->type = SQL_INSERT;

    // — 1단계: 테이블 이름 추출 —————
    //
    // sql = "INSERT INTO students VALUES (1, 'Kim', 20);"
    //      ^^^^^^^^^^^
    //      12글자 (공백 포함)
    //
    // strlen("INSERT INTO ") = 12이므로
    // sql + 12 = "students VALUES (1, 'Kim', 20);"
    //
    const char* p = sql + strlen("INSERT INTO ");

    // p에서 공백이나 '('를 만날 때까지 = 테이블 이름

```

```

int i = 0;
while (*p && !isspace(*p) && *p != '(') {
    result->table_name[i++] = *p++;
}
result->table_name[i] = '\\0';
// result->table_name = "students"

// — 2단계: VALUES 위치 찾기 —————
//
// strcasestr: 대소문자 무시하고 "VALUES" 찾기
// "values", "Values" 등도 다 찾아줌
//
const char* values_pos = strcasestr(sql, "VALUES");
if (!values_pos) return -1; // VALUES 없으면 에러

// — 3단계: 괄호 안의 값 추출 —————
//
// '(' 찾기
const char* paren_open = strchr(values_pos, '(');
if (!paren_open) return -1;
paren_open++; // '(' 다음 칸으로 이동
// paren_open = "1, 'Kim', 20);"

// ')' 찾기
const char* paren_close = strchr(paren_open, ')');
if (!paren_close) return -1;

// '('와 ')' 사이 내용을 복사 (strtok이 원본을 변경하니까)
char buf[MAX_SQL_LEN];
int len = paren_close - paren_open;
strncpy(buf, paren_open, len);
buf[len] = '\\0';
// buf = "1, 'Kim', 20"

// — 4단계: 쉼표로 분리 —————
//
result->value_count = 0;
char* token = strtok(buf, ",");

while (token != NULL) {
    char* trimmed = trim(token); // 공백 제거

```

```

// 작은따옴표 제거: 'Kim' → Kim
if (trimmed[0] == '\\') {
    trimmed++; // 앞 따옴표 건너뛰기
    trimmed[strlen(trimmed) - 1] = '\\0'; // 뒤 따옴표 제거
}

strcpy(result->values[result->value_count], trimmed);
result->value_count++;

token = strtok(NULL, ","); // 다음 값
}

return 0; // 성공
}

```

INSERT 파싱 단계별 시각화: 원본: "INSERT INTO students VALUES (1, 'Kim', 20);" 1단계 테이블 이름: INSERT INTO [students] VALUES (1, 'Kim', 20); L p가 여기서 시작 → 공백 만날 때까지 읽음 2단계 VALUES 찾기: INSERT INTO students [VALUES] (1, 'Kim', 20); L strcasestr로 위치 찾음 3단계 괄호 안 추출: VALUES ([1, 'Kim', 20]); L buf에 복사됨 4단계 쉼표로 분리: "1, 'Kim', 20" → strtok(",")

1	'Kim'	20
---	-------	----

← trim + 따옴표 제거 후

values[0] values[1] values[2]

SELECT 파싱 — 한 줄씩 해부

```

static int parse_select(const char* sql, ParsedSQL* result) {
    result->type = SQL_SELECT;

    // — 1단계: "SELECT "와 "FROM" 사이 = 컬럼 목록 —
    //
    // sql = "SELECT * FROM students;"
    //      ^^^^^^ ^^^^
    //      7글자   FROM 위치
    //
    const char* col_start = sql + strlen("SELECT ");
    // col_start = "* FROM students;"

```

```

const char* from_pos = strcasestr(sql, "FROM");
if (!from_pos) return -1;

// col_start부터 from_pos까지 = 컬럼 부분
char cols_buf[MAX_SQL_LEN];
int len = from_pos - col_start;
strncpy(cols_buf, col_start, len);
cols_buf[len] = '\0';
// cols_buf = "* " 또는 "name, age " 등

// — 2단계: 컬럼 파싱 (쉼표로 분리) —
//
result->column_count = 0;
char* token = strtok(trim(cols_buf), ",");
while (token) {
    strcpy(result->columns[result->column_count], trim(token));
    result->column_count++;
    token = strtok(NULL, ",");
}
// SELECT * → columns[0] = "*", column_count = 1
// SELECT name, age → columns[0]="name", columns[1]="age"

// — 3단계: 테이블 이름 추출 —
//
const char* table_start = from_pos + strlen("FROM ");
while (isspace(*table_start)) table_start++; // 혹시 모를 공백

int ti = 0;
while (*table_start && !isspace(*table_start) && *table_start != ';' ) {
    result->table_name[ti++] = *table_start++;
}
result->table_name[ti] = '\0';
// table_name = "students"

return 0;
}

```

SELECT 파싱 단계별 시각화: 원본: **"SELECT * FROM students;"** 1단계 컬럼 추출: SELECT **[*]** FROM students; ↳ "SELECT "와 "FROM" 사이 2단계 컬럼 분리: "*" → columns[0] = "*" 3단계 테이블

이름: SELECT * FROM **[students]**; L "FROM " 다음부터 공백이나 ' ; '까지

메인 파싱 함수 (진입점)

```
// 외부에서 호출하는 유일한 함수
// SQL 문자열을 받아서 ParsedSQL에 결과를 채운다.
// 성공하면 0, 실패하면 -1 반환.

int parse_sql(const char* sql, ParsedSQL* result) {
    // 구조체 초기화 (모든 값을 0으로)
    memset(result, 0, sizeof(ParsedSQL));

    // 앞쪽 공백 건너뛰기
    while (isspace(*sql)) sql++;

    // SQL 종류 판별 후 해당 파서 호출
    if (strncasecmp(sql, "INSERT INTO", 11) == 0) {
        return parse_insert(sql, result);
    }
    if (strncasecmp(sql, "SELECT", 6) == 0) {
        return parse_select(sql, result);
    }

    // 어느 것도 아니면 UNKNOWN
    result->type = SQL_UNKNOWN;
    return -1;
}
```

Q. memset이 뭔가요?

memset(주소, 값, 크기) = 메모리를 특정 값으로 채운다. memset(result, 0, sizeof(ParsedSQL)) 은 result 구조체의 모든 바이트를 0으로 초기화한다는 뜻이다. 이전에 남아있던 쓰레기 값을 깨끗이 지우는 것.

storage.c — 파일 기반 DB

각 테이블을 CSV 파일로 저장한다. INSERT = 파일에 쓰기, SELECT = 파일에서 읽기.

Row 구조체 (types.h에 추가)

binary 저장은 구조체를 통째로 파일에 쓰는 것이다. 그래서 먼저 "1행(Row)의 모양"을 정의해야 한다.

```
// types.h에 추가할 내용
#define MAX_ROW_COLS 32    // 한 행 최대 컬럼 수
#define COL_SIZE      256  // 각 컬럼 값 최대 크기

// 한 행(Row)을 표현하는 구조체
// 이 구조체가 통째로 binary 파일에 저장된다
typedef struct {
    int col_count;           // 이 행의 컬럼 개수
    char cols[MAX_ROW_COLS][COL_SIZE]; // 각 컬럼의 값
} Row;
```

왜 고정 크기 구조체인가?

Row 구조체의 크기는 항상 일정하다 (sizeof(Row) = 고정값).

덕분에 파일에서 N번째 행을 바로 찾아갈 수 있다:

```
fseek(fp, N * sizeof(Row), SEEK_SET)
```

CSV에서는 각 줄 길이가 달라서 이게 불가능하다. 이것이 binary 저장의 핵심 장점.

storage.h

```
#ifndef STORAGE_H
#define STORAGE_H

#include "types.h"
```

```

int storage_insert(const char* table_name,
                  char values[][MAX_VALUE_LEN],
                  int value_count);

int storage_select(const char* table_name);

#endif

```

storage.c 전체 코드 + 상세 설명 (binary 버전)

```

#include <stdio.h>
#include <string.h>
#include "types.h"
#include "storage.h"

// 테이블 이름으로 파일 경로 만들기
// "students" → "data/students.db"
static void get_table_path(const char* table_name, char* path) {
    sprintf(path, "data/%s.db", table_name);
}

// — INSERT 구현 (binary write) —
int storage_insert(const char* table_name,
                  char values[][MAX_VALUE_LEN],
                  int value_count)
{
    char path[256];
    get_table_path(table_name, path);

    // "ab" = append + binary 모드
    // "a" = 기존 내용 뒤에 추가
    // "b" = binary 모드 (텍스트 변환 하지 않음)
    FILE* fp = fopen(path, "ab");
    if (!fp) {
        fprintf(stderr, "Error: Cannot open table '%s'\n", table_name);
        return -1;
    }
}

```



```

// 1. Row 구조체를 0으로 초기화
Row row;
memset(&row, 0, sizeof(Row));

// 2. 값들을 Row 구조체에 채우기
row.col_count = value_count;
for (int i = 0; i < value_count; i++) {
    strncpy(row.cols[i], values[i], COL_SIZE - 1);
    // strncpy: 최대 COL_SIZE-1 글자만 복사 (버퍼 오버플로 방지)
}

// 3. Row 구조체를 통째로 파일에 쓰기
fwrite(&row, sizeof(Row), 1, fp);
// fwrite(데이터주소, 1개크기, 개수, 파일)
// Row 구조체 1개를 binary로 파일에 씀

fclose(fp);
printf(" Inserted into '%s'\n", table_name);
return 0;
}

// — SELECT 구현 (binary read) —————
int storage_select(const char* table_name) {
    char path[256];
    get_table_path(table_name, path);

    // "rb" = read + binary 모드
    FILE* fp = fopen(path, "rb");
    if (!fp) {
        fprintf(stderr, " Error: Table '%s' not found\n", table_name);
        return -1;
    }

    Row row;
    int row_num = 0;

    // fread: 파일에서 sizeof(Row) 바이트씩 읽기
    // 읽은 개수가 1이면 성공, 0이면 파일 끝
    while (fread(&row, sizeof(Row), 1, fp) == 1) {
        printf(" Row %d: ", row_num++);
    }
}

```

```

// 각 컬럼 값을 | 로 구분해서 출력
for (int i = 0; i < row.col_count; i++) {
    if (i > 0) printf(" | ");
    printf("%s", row.cols[i]);
}
printf("\n");
}

if (row_num == 0) {
    printf(" (empty table)\n");
}

fclose(fp);
return 0;
}

```

binary 저장 동작 시각화 INSERT: Row 구조체를 통째로 파일에 쓴다 Row 구조체 (메모리):

col_count = 3 | | cols[0] = "1\0\0\0..." (256바이트) | | cols[1] = "Kim\0\0\0..." (256바이트) | | cols[2] = "20\0\0\0..." (256바이트) |

 fwrite() → 통째로 파일에 쓴다 ▼ data/students.db (binary 파일):

[Row 0의 바이트들] (sizeof(Row)) | | [Row 1의 바이트들] (sizeof(Row)) | | [Row 2의 바이트들] (sizeof(Row)) | ← 추가됨

SELECT: 파일에서 Row 구조체를 통째로 읽는다

 data/students.db → fread()로 sizeof(Row)씩 읽기

 [Row 0 bytes] | → | Row 0: 1 | Kim | 20 | | [Row 1 bytes] | → | Row 1: 2 | Lee | 22 | | [Row 2 bytes] | → | Row 2: 3 | Park | 21 |

 (binary 파일) (화면 출력)

Q. fwrite / fread가 뭔가요?

fwrite(주소, 크기, 개수, 파일) = 메모리의 내용을 그대로 파일에 쓴다.

fread(주소, 크기, 개수, 파일) = 파일의 내용을 그대로 메모리로 읽어온다.

fprintf/fgets는 텍스트(문자열)를 다루지만, fwrite/fread는 **바이트 덩어리**를 다룬다. 구조체, 정수, 배열 등 아무거나 가능. 대신 사람이 파일을 열어봐도 읽을 수 없다 (깨진 문자처럼 보임).

Q. "ab" 모드에서 b는 뭔가요?

b 는 binary 모드를 뜻한다. 텍스트 모드("a", "r")에서는 줄바꿈(\n)이 OS에 따라 변환될 수 있는데, binary 모드("ab", "rb")에서는 바이트를 있는 그대로 읽고 쓴다. 구조체를 파일에 저장할 때는 반드시 binary 모드를 써야 데이터가 깨지지 않는다.

CHAPTER 7

executor.c — 실행기

parser와 storage를 연결하는 다리 역할. 가장 짧고 간단하다.

executor.h

```
#ifndef EXECUTOR_H
#define EXECUTOR_H
#include "types.h"
int execute(ParsedSQL* parsed);
#endif
```

executor.c

```
#include <stdio.h>
#include "types.h"
#include "storage.h"
#include "executor.h"

int execute(ParsedSQL* parsed) {
    switch (parsed->type) {

        case SQL_INSERT:
```

```

        // ParsedSQL에서 테이블명, 값들, 개수를 꺼내서 storage에 전달
        return storage_insert(
            parsed->table_name,
            parsed->values,
            parsed->value_count
        );

    case SQL_SELECT:
        // 테이블명만 넘기면 됨
        return storage_select(parsed->table_name);

    default:
        fprintf(stderr, " Error: Unknown SQL type\n");
        return -1;
}
}

```

Q. 왜 executor를 따로 분리하나요? storage를 직접 호출하면 안 되나요?

지금은 간단해서 직접 호출해도 되지만, 나중에 WHERE 절이나 ORDER BY 같은 기능이 추가되면 executor에서 "파싱 결과를 해석해서 추가 처리"를 하게 된다. 예를 들어 WHERE age > 20이면 storage에서 전체를 읽은 다음 executor에서 필터링하는 식이다. 관심사의 분리(Separation of Concerns)라고 하는 설계 원칙이다.

CHAPTER 8

main.c — 대화형 CLI (REPL) + 전체 연결

이전 버전은 SQL 파일을 읽는 방식이었지만, 이번에는 프롬프트에서 직접 SQL을 입력하는 REPL 방식이다. 추가로, 인자가 있으면 파일 모드로도 동작하게 했다.

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>    // system(), exit()

```

```

#include <ctype.h>      // isspace()
#include "types.h"
#include "parser.h"
#include "executor.h"

// — 한 줄의 SQL을 처리하는 함수 —————
// REPL과 파일 모드 모두에서 공통으로 사용
void process_line(const char* line) {
    // 앞쪽 공백 건너뛰기
    while (isspace(*line)) line++;

    // 빈 줄이나 주석이면 무시
    if (*line == '\\0') return;
    if (strncmp(line, "--", 2) == 0) return;

    // 파싱
    ParsedSQL parsed;
    if (parse_sql(line, &parsed) != 0) {
        fprintf(stderr, " Parse error!\\n");
        return;
    }

    // 실행
    if (execute(&parsed) != 0) {
        fprintf(stderr, " Execution error!\\n");
    }
}

// — REPL (대화형 모드) —————
// Read-Eval-Print Loop
// 프롬프트를 띄우고 → 입력 받고 → 처리하고 → 반복
void run_repl() {
    char line[MAX_SQL_LEN];

    printf("SQL Processor v1.0\\n");
    printf("Type SQL commands. '.exit' to quit.\\n\\n");

    while (1) { // 무한 반복
        // 1. 프롬프트 출력
        printf("db> ");
        fflush(stdout); // 버퍼 비우기 (프롬프트 즉시 표시)
    }
}

```

```

// 2. 사용자 입력 읽기
if (fgets(line, sizeof(line), stdin) == NULL) {
    // Ctrl+D (EOF) 입력 시
    printf("\nBye!\n");
    break;
}

// 3. 줄바꿈 제거
line[strcspn(line, "\n")] = '\0';

// 4. 종료 명령 체크
char* trimmed = line;
while (isspace(*trimmed)) trimmed++;

if (strcmp(trimmed, ".exit") == 0 ||
    strcmp(trimmed, ".quit") == 0) {
    printf("Bye!\n");
    break;
}

// 5. SQL 처리
process_line(trimmed);
}
}

// — 파일 모드 (인자로 .sql 파일을 넘긴 경우) —
int process_file(const char* filename) {
    FILE* fp = fopen(filename, "r");
    if (!fp) {
        fprintf(stderr, "Error: Cannot open '%s'\n", filename);
        return -1;
    }

    char line[MAX_SQL_LEN];
    while (fgets(line, sizeof(line), fp)) {
        line[strcspn(line, "\n")] = '\0';
        printf("db> %s\n", line);
        process_line(line);
    }
}

```

```

    fclose(fp);
    return 0;
}

// — 프로그램 진입점 —————
int main(int argc, char* argv[]) {
    // data 폴더 생성
    system("mkdir -p data");

    if (argc >= 2) {
        // 인자가 있으면 → 파일 모드
        // ./sql_processor test.sql
        return process_file(argv[1]);
    } else {
        // 인자가 없으면 → 대화형 모드 (REPL)
        // ./sql_processor
        run_repl();
        return 0;
    }
}

```

REPL 동작 흐름: `run_repl()` | | | `"db> "` 프롬프트 출력 | `fgets(line, ..., stdin)` ← 사용자 입력 대기 | | | | `".exit"` → `break` (종료) | | | | SQL 문장 → `process_line()` | | | `parse_sql()` → 파싱 | | `execute()` → 실행 | | 다시 `"db> "` 프롬프트 (무한 반복)

Q. stdin이 뭔가요?

`stdin` 은 표준 입력(Standard Input)이다. 키보드에서 입력하는 것을 읽을 때 쓴다.

`fgets(line, size, stdin)` = "키보드에서 한 줄 읽어서 line에 저장해"

파일에서 읽을 때 `fgets(line, size, fp)` 쓰는 것과 형태가 동일하다. `stdin` 자체가 FILE* 타입이다.

Q. fflush(stdout)는 왜 쓰나요?

`printf` 의 출력은 버퍼에 쌓여 있다가 한꺼번에 화면에 나온다. 줄바꿈(\n)이 있으면 자동으로 비워지는데, `"db> "` 에는 줄바꿈이 없어서 프롬프트가 바로 안 나올 수 있다. `fflush(stdout)` 는 "버퍼에 쌓인 걸 지금 당장 화면에 쏟

아내"라는 뜻이다.

Q. while(1)은 안 위험한가요?

`while(1)` 은 무한 반복이다. REPL 프로그램에서는 사용자가 종료할 때까지 계속 돌아야 하니까 무한 반복이 맞다.
`.exit` 입력 시 `break` 로 빠져나오고, Ctrl+D (EOF) 시에도 빠져나온다.

Q. continue가 뭔가요?

반복문에서 `continue` 를 만나면 남은 코드를 건너뛰고 다음 반복으로 넘어간다. 빈 줄이나 주석을 만났을 때 파싱/실행을 건너뛰고 다음 입력을 읽으라는 뜻이다.

CHAPTER 9

Makefile — 빌드 자동화

매번 `gcc main.c parser.c executor.c storage.c -o sql_processor` 이렇게 치기 귀찮으니까, `make` 한 단어로 빌드하게 해주는 파일이다.

```
CC = gcc                # 사용할 컴파일러
CFLAGS = -Wall -Wextra -g # 컴파일 옵션
                        # -Wall: 경고 다 보여줘
                        # -Wextra: 추가 경고도
                        # -g: 디버깅 정보 포함

TARGET = sql_processor  # 만들 실행파일 이름
SRCS = main.c parser.c executor.c storage.c

# 기본 명령: make 치면 이게 실행됨
all: $(TARGET)
```



```

# 실행파일 만드는 규칙
$(TARGET): $(SRCS)
    $(CC) $(CFLAGS) -o $@ $^

# 정리
clean:
    rm -f $(TARGET) test_runner
    rm -rf data/

# 기능 테스트 (파일 모드로 실행)
test: $(TARGET)
    @echo "=== Test Start ==="
    @rm -rf data/ && mkdir -p data
    @echo "INSERT INTO students VALUES (1, 'Kim', 20);" > /tmp/test.sql
    @echo "INSERT INTO students VALUES (2, 'Lee', 22);" >> /tmp/test.sql
    @echo "SELECT * FROM students;" >> /tmp/test.sql
    ./$(TARGET) /tmp/test.sql
    @echo "=== Test Done ==="

# 단위 테스트
unit_test: parser.c storage.c executor.c test.c
    $(CC) $(CFLAGS) -o test_runner $^
    ./test_runner

.PHONY: all clean test unit_test

```

사용법:

make → 빌드

./sql_processor → 대화형 모드 (REPL) 실행

make test → 빌드 + 파일 모드 테스트

make unit_test → 단위 테스트 실행

make clean → 실행파일, 데이터 폴더 삭제

Makefile 주의: 명령어 앞의 들여쓰기는 반드시 탭(Tab)이어야 한다. 스페이스로 하면 에러 남. 에디터 설정 확인 할 것.

테스트 전략

발표할 때 테스트 과정을 보여주면 완성도가 높아 보인다. 테스트는 크게 2종류:

1. 단위 테스트 (test.c)

개별 함수가 제대로 동작하는지 확인.

```
#include <stdio.h>
#include <string.h>
#include <assert.h>    // assert() 매크로
#include "types.h"
#include "parser.h"
#include "storage.h"

// — 파서 테스트 —————

void test_insert_basic() {
    ParsedSQL r;
    int ret = parse_sql("INSERT INTO users VALUES (1, 'Alice', 25);", &r);

    assert(ret == 0);                // 파싱 성공
    assert(r.type == SQL_INSERT);    // INSERT로 인식
    assert(strcmp(r.table_name, "users") == 0); // 테이블명 맞춤
    assert(r.value_count == 3);      // 값 3개
    assert(strcmp(r.values[0], "1") == 0);    // 첫 번째 값
    assert(strcmp(r.values[1], "Alice") == 0); // 두 번째 값
    assert(strcmp(r.values[2], "25") == 0);    // 세 번째 값

    printf(" PASS: test_insert_basic\n");
}

void test_select_all() {
    ParsedSQL r;
    int ret = parse_sql("SELECT * FROM products;", &r);
```

```

    assert(ret == 0);
    assert(r.type == SQL_SELECT);
    assert(strcmp(r.table_name, "products") == 0);
    assert(strcmp(r.columns[0], "*") == 0);
    assert(r.column_count == 1);

    printf(" PASS: test_select_all\n");
}

void test_case_insensitive() {
    ParsedSQL r;
    // 소문자로 써도 동작해야 함
    int ret = parse_sql("insert into users values (1, 'Bob', 30);", &r);
    assert(ret == 0);
    assert(r.type == SQL_INSERT);
    printf(" PASS: test_case_insensitive\n");
}

void test_unknown_sql() {
    ParsedSQL r;
    int ret = parse_sql("DROP TABLE users;", &r);
    assert(ret == -1); // 파싱 실패해야 함
    assert(r.type == SQL_UNKNOWN); // UNKNOWN이어야 함
    printf(" PASS: test_unknown_sql\n");
}

// — 스토리지 테스트 —————

void test_insert_and_select() {
    // 테스트용 데이터 삭제 후 시작
    system("mkdir -p data && rm -f data/test_table.db");

    char v1[][MAX_VALUE_LEN] = {"1", "TestName", "99"};
    int ret = storage_insert("test_table", v1, 3);
    assert(ret == 0);

    // binary 파일이 생성되었는지 확인
    FILE* fp = fopen("data/test_table.db", "rb");
    assert(fp != NULL);

    // Row 구조체로 읽어서 값 검증

```

```

Row row;
fread(&row, sizeof(Row), 1, fp);
assert(row.col_count == 3);
assert(strcmp(row.cols[0], "1") == 0);
assert(strcmp(row.cols[1], "TestName") == 0);
assert(strcmp(row.cols[2], "99") == 0);
fclose(fp);

// 정리
system("rm -f data/test_table.db");
printf(" PASS: test_insert_and_select\n");
}

// — 메인 —————

int main() {
    printf("=== Parser Tests ===\n");
    test_insert_basic();
    test_select_all();
    test_case_insensitive();
    test_unknown_sql();

    printf("\n=== Storage Tests ===\n");
    test_insert_and_select();

    printf("\n✓ All tests passed!\n");
    return 0;
}

```

Q. assert()가 뭔가요?

assert(조건) = 조건이 참이면 아무 일도 안 일어남. 거짓이면 프로그램이 즉시 멈추고 에러 메시지 출력. 테스트 할 때 "이 값이 이거여야 한다"를 검증하는 가장 간단한 방법이다.

2. 대화형 테스트 시나리오

REPL 모드에서 직접 테스트할 수 있다:

```

$ ./sql_processor
db> insert into students values (1, 'Kim', 20);
    Inserted into 'students'
db> insert into students values (2, 'Lee', 22);
    Inserted into 'students'
db> select * from students;
    Row 0: 1 | Kim | 20
    Row 1: 2 | Lee | 22
db> select * from nonexistent_table;
    Error: Table 'nonexistent_table' not found
db> insert into courses values (101, 'Database', 3);
    Inserted into 'courses'
db> select * from courses;
    Row 0: 101 | Database | 3
db> .exit
Bye!

```

또는 파일 모드로도 테스트 가능 ([make test](#)).

CHAPTER 11

엣지 케이스 & 에러 처리

이것들을 처리하면 완성도가 확 올라간다.

케이스	예시	처리 방법
공백 많음	INSERT INTO t VALUES (1 , 'a');	trim()으로 공백 제거
대소문자 혼합	insert Into t values (1);	strncasecmp/strcasestr 사용
세미콜론 없음	SELECT * FROM t	',' 없어도 파싱되게 구현
빈 줄	(아무것도 없는 줄)	main.c에서 continue로 건너뛸

주석	-- 이것은 주석	"--"로 시작하면 건너뛴
없는 테이블 SELECT	SELECT * FROM ghost;	fopen 실패 → 에러 메시지
빈 테이블 SELECT	(데이터 없는 테이블)	"(empty table)" 출력
지원 안 하는 SQL	DROP TABLE t;	SQL_UNKNOWN → 에러 메시지

에러 처리 원칙:

에러가 나도 프로그램이 죽지 않고 다시 프롬프트로 돌아와야 한다. process_line()에서 에러 메시지만 출력하고 return하면 REPL 루프가 계속 돈다. 한 줄이 실패해도 다음 SQL을 계속 입력할 수 있어야 한다.

CHAPTER 12

차별화 추가 구현

기본을 다 했으면 아래에서 골라서 추가 구현하면 다른 팀과 차이를 만들 수 있다.

추천 1: WHERE 절 (가성비 최고)

```
SELECT * FROM students WHERE age > 20;
```

구현 방법: storage_select에서 전체 행을 읽되, WHERE 조건에 맞는 행만 출력. ParsedSQL에 where_column, where_operator, where_value 필드를 추가하면 된다.

추천 2: DELETE

```
DELETE FROM students WHERE id = 1;
```

구현 방법: binary 파일을 fread로 Row씩 읽으면서, 조건에 안 맞는 Row만 임시 파일에 fwrite하고, 원본을 임시 파일로 교체(rename).

추천 3: 특정 컬럼 SELECT

```
SELECT name, age FROM students;
```

구현 방법: 스키마 정보(컬럼 순서)를 파일 첫 줄에 저장하고, 해당 컬럼 인덱스만 출력.

추천 4: 예쁜 테이블 출력

기존:

Row 0: 1, Kim, 20

Row 1: 2, Lee, 22

개선:

```
+-----+-----+-----+
| id | name | age |
+-----+-----+-----+
|  1 | Kim  |  20 |
|  2 | Lee  |  22 |
+-----+-----+-----+
2 rows selected.
```

이건 비교적 쉬운데 보여주기(데모) 효과가 크다.

발표 준비 체크리스트

목요일 발표 4분 + QnA 1분. README.md 기준으로 발표.

README.md에 넣을 내용

SQL Processor

C로 구현한 대화형 SQL 처리기 (binary 파일 기반 DB)

실행 방법

```
```bash
make
./sql_processor # 대화형 모드
./sql_processor test.sql # 파일 모드
```
```

지원하는 SQL

- INSERT INTO table VALUES (...)
- SELECT * FROM table

아키텍처

입력(CLI) → 파싱(parser) → 실행(executor) → 저장(binary storage)

데이터 저장

- 각 테이블은 binary 파일로 관리 (data/테이블명.db)
- Row 구조체를 fwrite/fread로 직접 저장/읽기

테스트

```
```bash
make test # 기능 테스트
make unit_test # 단위 테스트
```
```

추가 구현

- (WHERE, DELETE 등 추가한 거)

데모 시나리오 (4분)

- 1 프로젝트 소개 (30초) — 뭘 만들었는지, 대화형 CLI + binary 저장 아키텍처 설명
- 2 기본 데모 (1분 30초) — REPL에서 INSERT 여러 개 → SELECT 확인 → hexdump로 binary 파일 구조 보여주기

3 테스트 데모 (1분) — make unit_test 실행 → 파서/스토리지 테스트 결과 보여주기 → 옛지 케이스 처리

4 추가 구현 / 핵심 코드 설명 (1분) — 차별화 포인트 데모 + 파싱/binary 저장 로직 설명

발표 때 꼭 기억할 것:

"AI가 작성한 코드를 완벽히 내 것으로 소화해 그 원리를 설명할 수 있어야 합니다."

QnA에서 "이 코드가 뭘 하는 건가요?" 질문이 나올 수 있다. 특히 parser.c의 파싱 로직은 확실히 이해해야 한다.

QnA 대비 질문들

Q. strtok은 어떻게 동작하나요?

첫 호출 시 원본 문자열을 받고, 구분자(쉼표)를 null 문자로 바꿔서 첫 토큰을 반환한다. 이후 NULL을 넘기면 이전 위치에서 이어서 다음 토큰을 반환한다. 원본 문자열이 변경되기 때문에 복사본을 사용했다.

Q. 왜 binary 포맷으로 저장하나요?

실제 DB와 유사한 방식이다. 고정 크기 Row 구조체를 binary로 저장하면 N번째 행을 바로 찾아갈 수 있고 (fseek), 파싱 과정 없이 fread 한 번이면 데이터가 메모리에 올라온다. CSV보다 빠르고 데이터 종속성이 유지된다.

Q. fwrite/fread와 fprintf/fgets의 차이는?

fprintf/fgets는 텍스트(문자열)를 다루고, fwrite/fread는 바이트 덩어리를 그대로 읽고 쓴다. 구조체를 통째로 저장할 때는 fwrite/fread를 써야 하고, 반드시 "b" (binary) 모드로 파일을 열어야 데이터가 깨지지 않는다.

Q. 파일에 직접 저장하는 방식의 한계는?

DELETE나 UPDATE 시 전체 파일을 다시 써야 하고, 동시 접근이 안 되며, 데이터가 많아지면 SELECT가 느려진다 (매번 전체 읽기). 실제 DB는 인덱스, 페이지 단위 관리, 트랜잭션 등으로 이 문제를 해결한다.

Q. 왜 대화형(REPL) 방식으로 만들었나요?

실제 MySQL, SQLite도 대화형 셸을 제공한다. 사용자가 즉석에서 SQL을 입력하고 결과를 바로 확인할 수 있어서 테스트와 디버깅이 편하다. 파일 모드도 함께 지원해서 자동화된 테스트도 가능하게 했다.

이 문서의 모든 코드를 이해하면 수요 코딩회 준비 끝.

화이팅!